Hes·so
Haute Ecole Spécialisée
de Suisse occidentale
Fachhochschule Westschweiz
University of Applied Sciences and Arts
Western Switzerland

Linux Embedded Systems

Lab Report

Student	Sylvan Arnold
Program	Computer Science
Academic year	2026
Class	MA_CSEL1
Repository	https://github.com/SylvanArnold/csel-workspace

HES-SO

1. Introduction

This is the second report of the Linux Embedded Systems course. In this document, I present my work on the exercises of the second part of the course, which covers file systems, multiprocessing and scheduling, performance analysis and optimization.

The link to the repository with my solutions: `https://github.com/SylvanArnold/csel-workspace`

2. File Systems

I analysed the `silly_led_control.c` code. I understand why it consumes 100% of the CPU. It is constantly reading the clock to count the elapsed time instead of using a timer and sleeping.

In my implementation, push buttons are connected to GPIO0, GPIO2, and GPIO3. I created an epoll instance and added the buttons file descriptors to it. Then, I used `epoll_wait` to wait for an event on any of the buttons. When an event is triggered, I read the button index from the event.data field.

I separated my code into two threads: one for the buttons and one for the LED blinking. The button thread waits for an event on the buttons and updates the blink period accordingly. The LED thread blinks the LED with the current blink period using `timerfd`. The blink period is a shared variable between the two threads, so I protected it by a mutex. The updated blink period is printed in the console and sent to the syslog.

I checked CPU usage of my implementation with `top` and it is at most 1.3%.

My /var/log/messages:

```
Jan  1 00:50:29 csel user.info better_led[261]: New period: 500 ms
Jan  1 00:54:10 csel user.info better_led[261]: New period: 450 ms
Jan  1 00:54:11 csel user.info better_led[261]: New period: 400 ms
Jan  1 00:54:11 csel user.info better_led[261]: New period: 350 ms
Jan  1 00:54:12 csel user.info better_led[261]: New period: 300 ms
Jan  1 00:54:12 csel user.info better_led[261]: New period: 250 ms
```

3. Multiprocessing and scheduling

3.1. Exercise 1

I created a simple c program that creates a socket pair and then makes a fork. Parent and child processes each take one end of the socket pair. Parent takes CPU 0 and child takes CPU 1 with the `sched_setaffinity` call. Then they subscribe to SIGHUP, SIGINT, SIGQUIT, SIGABRT, SIGTERM signals with a handler that simply prints the signal received in the console.

Then the child process sends a series of messages to the parent that logs them. The final message is an exit message and then the child exits. The parent process waits for the child to exit and then it exits too.

At first, I didn't understand that both parent and child continue execution from the point immediately after the fork() call.

I ran the program and then in another terminal I sent a SIGINT signal to the parent process with `kill -SIGINT <parent_pid>`. The signal was ignored as expected:

```
Starting program
[PID 264] Affinity pinned to core 0.
[PARENT PID 264] Waiting for messages from child (PID 265)...

[PID 265] Affinity pinned to core 1.
[CHILD PID 265] Started on core 1.
[CHILD PID 265] Message sent: "Hello from the child!"
[PARENT PID 264] Message received: "Hello from the child!"
[CHILD PID 265] Message sent: "How are you, parent?"
[PARENT PID 264] Message received: "How are you, parent?"
[CHILD PID 265] Message sent: "Bybye"
[PARENT PID 264] Message received: "Bybye"
[PID 264] Signal SIGINT received and ignored.
[CHILD PID 265] Message sent: "exit"
[PARENT PID 264] Message received: "exit"
[CHILD PID 265] Terminated.

[PARENT PID 264] "exit" message received — shutting down.
[PARENT PID 264] Child exited (status 0). Goodbye!
```

3.2. Exercise 2

I created a small program that allocates 25 MB of memory and zeroes it. Then I created a cgroup with a 20 MB memory limit and added the current shell process to the cgroup. I ran the program:

```
./build/main
Killed
```

The program was killed. Then I tried to allocate only 15 MB of memory:

```
./build/main
Allocation SUCCESS (15728640 bytes)
```

The allocation succeeded, as expected.

3.2.1. Question 1: echo > effects

`echo $$ > ...` on a cgroup file adds the current process (the shell) to the targeted cgroup.

3.2.2. Question 2: memory limit reached effects

When the memory limit is reached, the OOM killer is triggered and kills one or multiple processes in the cgroup to free memory.

We can change this setting by disabling the OOM killer for the cgroup by writing `1` to the `memory.oom_control` file of the cgroup. In this case, when the memory limit is reached, the process that tries to allocate memory will receive an `ENOMEM` error instead of being killed.

3.2.3. Question 3: cgroup memory usage monitoring

You can use the `/sys/fs/cgroup/memory/memory.stat` file to monitor the memory usage of a cgroup:

```
# cat /sys/fs/cgroup/memory/mem/memory.stat
cache 1118208
rss 397312
rss_huge 0
shmem 0
mapped_file 544768
dirty 0
writeback 0
swap 0
pgpgin 11939
pgpgout 11562
pgfault 10475
pgmajfault 16
inactive_anon 335872
active_anon 4096
inactive_file 1093632
active_file 0
unevictable 0
hierarchical_memory_limit 20971520
hierarchical_memsw_limit 9223372036854771712
total_cache 1118208
total_rss 397312
total_rss_huge 0
total_shmem 0
total_mapped_file 544768
total_dirty 0
total_writeback 0
total_swap 0
total_pgpgin 11939
total_pgpgout 11562
total_pgfault 10475
total_pgmajfault 16
total_inactive_anon 335872
total_active_anon 4096
total_inactive_file 1093632
total_active_file 0
total_unevictable 0
```

Current rss memory usage is 0.39Mb and cache usage is 1.11Mb. The result is coherent since no program is currently running in the cgroup.

3.3. Exercise 3

For this exercise I created a small c code that does a fork in the beginning and then enters in an infinite loop to consume all the CPU. Then I ran the provided commands to create the cgroups:

```
mkdir /sys/fs/cgroup/cpuset
mount -t cgroup -o cpu,cpuset cpuset /sys/fs/cgroup/cpuset
mkdir /sys/fs/cgroup/cpuset/high
mkdir /sys/fs/cgroup/cpuset/low
echo 3 > /sys/fs/cgroup/cpuset/high/cpuset.cpus
echo 0 > /sys/fs/cgroup/cpuset/high/cpuset.mems
echo 2 > /sys/fs/cgroup/cpuset/low/cpuset.cpus
echo 0 > /sys/fs/cgroup/cpuset/low/cpuset.mems
```

3.3.1. Question 1: four last lines usefulness

The `high` and `low` groups that we created are subgroups of the parent cgroup. These subgroups do not inherit the CPU and memory configurations from the parent and must be initialized manually. If we don't do it, they will be considered as invalid by the kernel because no resources are allocated to them.

In our case, for the `high` group, we set CPU 3 and memory node 0. For the `low` group, we set CPU 2 and the same memory node as the high group.

3.3.2. Question 2: start the app in the two groups

I opened 3 shells: one in the `high` group, one in the `low` group, and one to run the `htop` command to monitor the CPU usage. I ran the app in the `high` group and in the `low` group simultaneously. You can see the result in Figure 1 below:

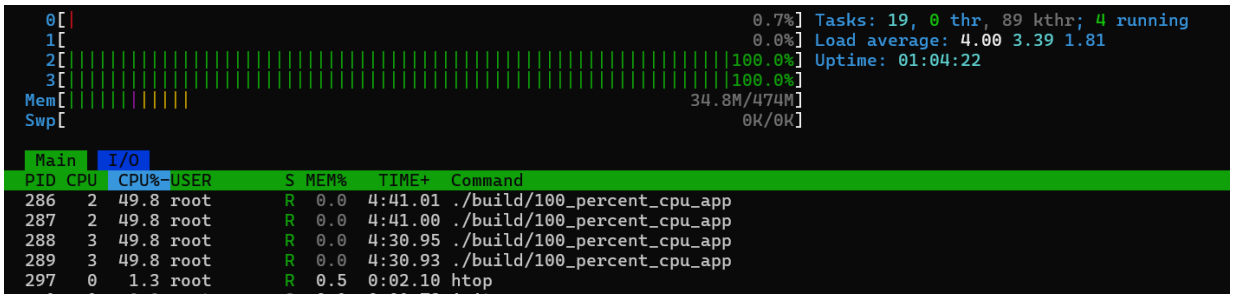

Figure 1: screenshot of running processes

We see that CPUs 2 and 3 are fully used by the running apps. Each app has two processes that take 50% of the CPU each, which is coherent since each app is doing a fork.

3.3.3. Question 3: cpu repartition on two tasks

To have on the same CPU, one task using 25% and the other using 75% of the CPU, using cgroups, I did this:

First, create two cgroups:

```
mkdir /sys/fs/cgroup/cpuset
mount -t cgroup -o cpu,cpuset cpuset /sys/fs/cgroup/cpuset
mkdir /sys/fs/cgroup/cpuset/group1
mkdir /sys/fs/cgroup/cpuset/group2
```

Then set both groups on cpu 3 and memory node 0:

```
echo 3 > /sys/fs/cgroup/cpuset/group1/cpuset.cpus
echo 0 > /sys/fs/cgroup/cpuset/group1/cpuset.mems
echo 3 > /sys/fs/cgroup/cpuset/group2/cpuset.cpus
echo 0 > /sys/fs/cgroup/cpuset/group2/cpuset.mems
```

Then, use the `cpu.shares` file to set the bandwidth for each group:

```
echo 256 > /sys/fs/cgroup/cpuset/group1/cpu.shares
echo 768 > /sys/fs/cgroup/cpuset/group2/cpu.shares
```

(the values are relative to 1024)

Finally, I started my app in each group. You can see the result in Figure 2 below:

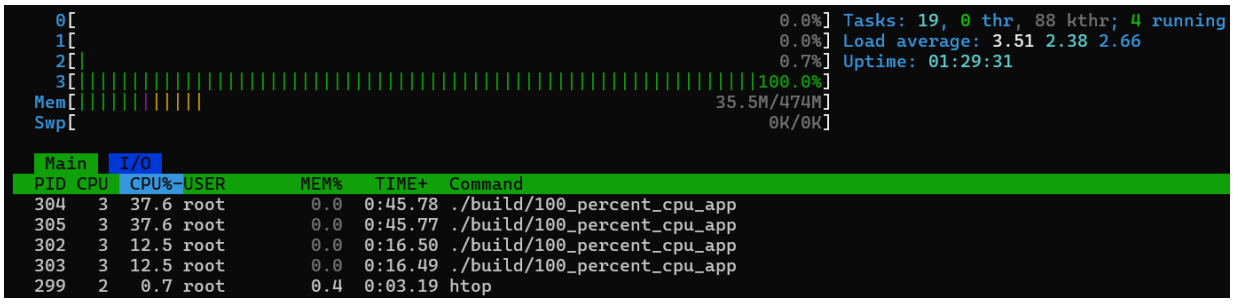

Figure 2: screenshot of cpu repartition

We see 4 processes running on CPU 3: two from the first group that take 12.5% of the CPU each, and two from the second group that take 37.6% of the CPU each.

4. Performance Analysis and Optimization

4.1. Setup

I've modified buildroot menuconfig to include binutils, rebuilt the image, updated the rootfs and rebuilt `perf` as asked in the instructions

4.2. Ex01 cache misses fix

I ran the `perf stat ./ex1` command and got the following result:

```
Performance counter stats for './ex1':

          37225.34 msec task-clock                #    0.999 CPUs utilized
                20      context-switches          #    0.537 /sec
                 0      cpu-migrations            #    0.000 /sec
             48867      page-faults               #    1.313 K/sec
       30375599323      cycles                    #    0.816 GHz
        1667024369      instructions              #    0.05  insn per cycle
         269096909      branches                  #    7.229 M/sec
            996300      branch-misses             #    0.37% of all branches

      37.252207351 seconds time elapsed

      36.526202000 seconds user
       0.323629000 seconds sys
```

The program took 37 seconds to finish. We see how many instructions were executed, how many CPU cycles it took, the context switches,... but we don't have information about cache misses.

I ran the `perf stat -e cache-misses ./ex1` command to get cache-miss information:

```
Performance counter stats for './ex1':

        406644166      cache-misses

    37.308410143 seconds time elapsed

    36.551646000 seconds user
      0.288202000 seconds sys
```

I analysed the code to understand why there are so many cache misses. The program is doing a column-wise access to a 2D array, which is not cache-friendly since the data is stored in row-wise order. So I inverted the `i` and `j` indexes in the access loop and got this result:

```
 Performance counter stats for './ex1':

          1213872      cache-misses

       2.461410335 seconds time elapsed

       2.170178000 seconds user
       0.233748000 seconds sys
```

We see a huge performance improvement, with way fewer cache misses and lower execution time.

4.3. Describe capturable events

instructions: counts the number of instructions executed by the program. cache-misses: counts the number of times the CPU failed to find data in the cache and had to fetch it from main memory. branch-misses: counts the number of times the CPU mispredicted a branch result, which decreases pipelining efficiency. L1-dcache-load-misses: counts the number of times the CPU failed to find data in the L1 data cache and had to fetch it from the next level of cache or main memory. cpu-migrations: counts the number of times a process was moved from one CPU to another, which can cause performance degradation context-switches: counts the number of times the CPU switched from one process or thread to another. Too many context switches adds overhead and decreases performance.

4.4. Measure perf impact on program performance

I ran `time ./ex1` and got this result:

```
real    0m 2.46s
user    0m 2.19s
sys     0m 0.21s
```

And with `perf stat ./ex1`:

```
2.541872792 seconds time elapsed

2.216295000 seconds user
0.252450000 seconds sys
```

The execution time is slightly higher with `perf`, but the difference is not that big.

4.5. Ex02 analysis and optimization

This program fills an array with random values between 0 and 512. Then it sums all the values in the array that are below 256 threshold. And it does that 10000 times.

As the values inside the array are not sorted, the branch predictor that predicts if the value is below the threshold or not will not be efficient, which will cause a lot of branch misses.

Branch prediction enables the cpu to pre-load the next instructions in the pipeline, so when a branch is mispredicted, the pipeline has to be flushed and reloaded with the correct instructions, which causes a performance penalty.

The result of `perf stat ./ex2` before optimization:

```
Performance counter stats for './ex2':

        26174.85 msec task-clock                #    0.998 CPUs utilized
              19      context-switches          #    0.726 /sec
               0       cpu-migrations            #    0.000 /sec
              75      page-faults               #    2.865 /sec
      21358547563       cycles                    #    0.816 GHz
      14768622436       instructions              #    0.69  insn per cycle
        988535403       branches                  #   37.767 M/sec
        327863164       branch-misses             #   33.17% of all branches

    26.229924221 seconds time elapsed

    26.120242000 seconds user
     0.003976000 seconds sys
```

We have 33.17% of branch misses. To optimize the program, I used the provided sorting function that sorts the values in the array before summing them.

The result after optimization:

```
Performance counter stats for './ex2':

        23432.46 msec task-clock                #    0.998 CPUs utilized
              20      context-switches          #    0.854 /sec
               0       cpu-migrations            #    0.000 /sec
             107       page-faults               #    4.566 /sec
      19120754322       cycles                    #    0.816 GHz
      14818352704       instructions              #    0.77  insn per cycle
        997830308       branches                  #   42.583 M/sec
           813078       branch-misses             #    0.08% of all branches

    23.488982552 seconds time elapsed

    23.383443000 seconds user
     0.003990000 seconds sys
```

We have far fewer branch misses and execution time is slightly better. The sorting function takes some time to execute but it is compensated by the fact that we do the sum 10000 times. If we did the sum only once, this optimization would not be efficient.

4.6. Logs Apache parsing

I executed the perf record command and displayed the result with `perf report`. Here are the functions that consume the most CPU with the extended call graph:

```
- 25.44%  read-apache-log  read-apache-logs  [.] std::operator==<char>
  std::find<...>
  HostCounter::isNewHost
  HostCounter::notifyHost
  ApacheAccessLogAnalyzer::processFile
  main
- 19.23%  read-apache-log  read-apache-logs  [.]
__gnu_cxx::__normal_iterator<...>
  std::find<...>
  HostCounter::isNewHost
  HostCounter::notifyHost
  ApacheAccessLogAnalyzer::processFile
  main
- 19.13%  read-apache-log  read-apache-logs  [.]
__gnu_cxx::__ops::_Iter_equals_val<...>
  std::find<...>
  HostCounter::isNewHost
  HostCounter::notifyHost
  ApacheAccessLogAnalyzer::processFile
  main
+  9.18%  read-apache-log  read-apache-logs  [.]
__gnu_cxx::__normal_iterator<...>
+  9.02%  read-apache-log  read-apache-logs  [.] std::__find_if<...>
+  5.06%  read-apache-log  read-apache-logs  [.]
std::__cxx11::basic_string<...>::size@plt
+  5.02%  read-apache-log  libc.so.6         [.] memcmp
```

I see that all these functions are related to the `isNewHost` function of the `HostCounter` class. This is the function to optimize.

Here is the code of the `isNewHost` function:

```
bool HostCounter::isNewHost(std::string hostname)
{
    return std::find(myHosts.begin(), myHosts.end(), hostname) ==
myHosts.end();
}
```

This function is iterating over the entire hosts vector each time to check if the hostname is already in the vector or not. This is not efficient since the vector can grow a lot and we will have to iterate over it many times.

I applied the suggested modifications in the instructions to use a `std::set` instead of a `std::vector` to store the hosts. The `std::set` is implemented as a balanced binary search tree, which allows for logarithmic time complexity for search operations.

After the optimization, I got this result:

```
Samples: 112  of event 'cpu-clock', Event count (approx.): 1493333296
  Overhead  Command          Shared Object        Symbol
-   8.04%  read-apache-log  libstdc++.so.6.0.29  [.]
std::__cxx11::basic_string<...>::compare
  std::operator< <...>
  std::less<std::__cxx11::basic_string<...>>::operator()
-   5.36%  read-apache-log  read-apache-logs     [.]
std::_Rb_tree<std::__cxx11::basic_string<...>, std::_...>
+ 4.46%  read-apache-log  libc.so.6            [.] cfree
+ 4.46%  read-apache-log  libc.so.6            [.] memcmp
+ 4.46%  read-apache-log  libstdc++.so.6.0.29  [.] 0x00000000000d9fe8
+ 4.46%  read-apache-log  read-apache-logs     [.]
std::_Rb_tree_node<std::__cxx11::basic_string<...>>
+ 4.46%  read-apache-log  read-apache-logs     [.]
std::less<std::__cxx11::basic_string<...>>::operator()
+ 4.46%  read-apache-log  read-apache-logs     [.] std::operator< <...>
+ 3.57%  read-apache-log  read-apache-logs     [.]
std::_Rb_tree<std::__cxx11::basic_string<...>, std::_...>
```

The overall performance is now much better. But I had even better results when I used a `std::unordered_set` instead of a `std::set`:

```
Samples: 75  of event 'cpu-clock', Event count (approx.): 999999975
  Overhead  Command          Shared Object          Symbol
+ 6.67%  read-apache-log  libstdc++.so.6.0.29  [.]
std::__cxx11::basic_string<...>::find_first_of
+ 5.33%  read-apache-log  libc.so.6            [.] cfree
+ 5.33%  read-apache-log  libstdc++.so.6.0.29  [.] memchr@plt
+ 4.00%  read-apache-log  libc.so.6            [.] malloc
+ 4.00%  read-apache-log  read-apache-logs     [.]
std::__detail::_Hashtable_base<std::__cxx11::basic_string<...>>
+ 2.67%  read-apache-log  libc.so.6            [.] 0x0000000000079104
+ 2.67%  read-apache-log  libc.so.6            [.] 0x0000000000085500
+ 2.67%  read-apache-log  libc.so.6            [.] 0x000000000008552c
+ 2.67%  read-apache-log  libstdc++.so.6.0.29  [.]
std::__cxx11::basic_string<...>::_M_construct<char*>
+ 2.67%  read-apache-log  libstdc++.so.6.0.29  [.] 0x00000000000d9fe8
+ 2.67%  read-apache-log  read-apache-logs     [.]
std::__detail::_Hash_code_base<std::__cxx11::basic_string<...>>
+ 1.33%  read-apache-log  [kernel.kallsyms]    [k]
el0_svc_common.constprop.0
+ 1.33%  read-apache-log  [kernel.kallsyms]    [k] filemap_read
```

Finally, I removed useless string cloning in the `isNewHost` and `notifyHost` functions, by passing the strings by reference and got this result:

```
Samples: 89  of event 'cpu-clock', Event count (approx.): 1186666637
  Overhead  Command          Shared Object        Symbol
+ 3.37%  read-apache-log  [kernel.kallsyms]    [k] __arch_copy_to_user
+ 3.37%  read-apache-log  [kernel.kallsyms]    [k] __do_softirq
+ 3.37%  read-apache-log  [kernel.kallsyms]    [k]
_raw_spin_unlock_irqrestore
+ 3.37%  read-apache-log  libstdc++.so.6.0.29  [.]
std::__cxx11::basic_string<...>::find_first_of
+ 3.37%  read-apache-log  libstdc++.so.6.0.29  [.] std::getline<char,
std::char_traits<char>, std::allocator<char>>
+ 3.37%  read-apache-log  libstdc++.so.6.0.29  [.]
std::istream::sentry::sentry
+ 3.37%  read-apache-log  read-apache-logs     [.]
std::_Hashtable<std::__cxx11::basic_string<...>, std::...>
+ 2.25%  read-apache-log  libc.so.6            [.] malloc
+ 2.25%  read-apache-log  libc.so.6            [.] 0x0000000000085500
+ 2.25%  read-apache-log  libc.so.6            [.] 0x0000000000085510
+ 2.25%  read-apache-log  libstdc++.so.6.0.29  [.] memchr@plt
+ 2.25%  read-apache-log  libstdc++.so.6.0.29  [.]
std::__cxx11::basic_string<...>::_M_create
+ 2.25%  read-apache-log  libstdc++.so.6.0.29  [.]
std::__cxx11::basic_string<...>::find_first_of
+ 2.25%  read-apache-log  read-apache-logs     [.] HostCounter::isNewHost
```

4.6.1. Question: how to measure interrupt latency and jitter

To have the best precision, we can use a hardware solution: with an oscilloscope that measures the time between the interrupt signal and the response signal.

In kernel space, we can create a small module that registers an interrupt handler on a pin and toggles another pin when the interrupt is triggered. Then we can use an oscilloscope to measure the time between the interrupt signal and the response signal.

In user space, we can do the same operation with a small application that uses `poll` to wait for an interrupt and toggle a GPIO pin when the interrupt is triggered.

To measure jitter, we do the measurement multiple times and calculate the standard deviation of the latency measurements.

5. Conclusion

During this laboratory, I learnt a lot about how to monitor and optimize a linux sytem. I particularly appreciated the use of `perf` to analyze the performance of a program and identify optimization opportunities. I also found interesting the use of cgroups to allocate ressources to processes.